M291 · SUPPORT · S11 FETCH

s11 — Afficher ses fiches avec fetch

Module ICT 291 · Développer des interfaces utilisateurs (UI)

Médiamaticien·ne CFC · Semaine 11 · 3 périodes de 45 minutes

Cette semaine, votre page va chercher `data.json` et en fait des cartes. Les cartes écrites à la main de la semaine 9 disparaissent : chaque fiche de `data.json` devient une carte, sans toucher au HTML.

> **NOTE**
> Deux fichiers. — Celui-ci est la version **cours** (on le parcourt ensemble). La version approfondie — plus de contexte, les pièges, la forme `await` et le lien avec votre projet — est le fichier `s11-fetch-avance.pdf`.

La scène

Sur le quai de la gare de Lausanne, l'écran des départs annonce : « IR 15, Genève-Aéroport, 14 h 12, voie 3, + 4 min ». Personne n'a écrit ce retard dans l'écran. L'écran demande les horaires à un ordinateur central, reçoit une liste de trains, et en fait une ligne par train.

Le jour où l'ordinateur central ne répond pas, l'écran n'affiche pas une liste vide : il affiche « Informations momentanément indisponibles ». Une liste vide ferait croire qu'aucun train ne part.

L'idée

Votre page fait la même chose, en petit. Elle contient la mise en page et la fonction qui fabrique une carte. Le contenu, lui, est dans `data.json`. Quand la page s'ouvre, elle **demande** ce fichier, le **lit**, puis **montre** une carte par fiche.

Rappel de la semaine 10 : le **JSON** est une façon d'écrire des données en texte, en fiches entre `{ }` rangées dans une pile entre `[ ]`, avec des étiquettes entre guillemets. `data.json` est un fichier de ce type.

Quatre mots suffisent pour cette semaine :

- un **serveur** : un programme qui donne des fichiers quand on les lui demande. GitHub Pages en est un. Sur votre ordinateur, l'extension **Live Server** de VS Code en joue le rôle ;
- **`fetch`** : l'instruction JavaScript qui demande un fichier au serveur. La demande s'appelle une **requête**, ce qui revient s'appelle la **réponse** ;
- le **code de statut** : le nombre que le serveur joint à sa réponse. `200` veut dire « voici le fichier », `404` veut dire « ce fichier n'existe pas » ;
- **`.then`** : « quand la réponse sera arrivée, fais ceci ». La réponse met un moment à venir, quelques millisecondes ou quelques secondes. La page n'attend pas : elle continue, et le `.then` s'exécute à l'arrivée. On dit que `fetch` est **asynchrone**.

Un exemple : les trois temps

Voici le code de Phare, l'application exemple du cours (les endroits calmes encore ouverts ce soir, près de chez soi) :

```
fetch("data.json")
  .then(function (reponse) {
    return reponse.json();
  })
  .then(function (donnees) {
    tousLesLieux = donnees;
    afficherLieux(tousLesLieux);
  });
```

Il se lit en trois temps :

Ligne	Temps	En français
`fetch("data.json")`	Demander	« Serveur, donne-moi `data.json`. »
`return reponse.json();`	Lire	« Lis le texte reçu comme du JSON. » Le résultat est la pile de fiches, comme en semaine 10.
`tousLesLieux = donnees;` puis `afficherLieux(tousLesLieux);`	Montrer	« Range les fiches dans la pile du haut du fichier, puis montre-les. »

`afficherLieux` est la fonction qui fabrique une carte par fiche. Elle utilise `createElement` (fabriquer un élément HTML vide) et `textContent` (y mettre un texte, qui s'affiche tel quel). Dans votre projet, c'est l'IA qui l'adapte à vos cartes ; vous vérifiez qu'elle montre bien vos étiquettes.

Le geste : regarder ce qui arrive

`fetch` ne fonctionne pas sur une page ouverte par un double-clic. L'adresse commence alors par `file://`, et le navigateur refuse de lire un autre fichier du disque. Ouvrez la page avec Live Server : clic droit sur `index.html` dans VS Code, puis *Open with Live Server*. L'adresse commence par `http://127.0.0.1:5500`.

Ensuite, ouvrez les outils du navigateur (touche F12 ; sur Mac, Cmd+Option+I) et l'onglet **Réseau** (Network). Rechargez la page. Chaque fichier demandé y a sa ligne : `index.html`, `style.css`, `app.js`, puis `data.json`. La colonne **État** donne le code de statut. Un clic sur la ligne `data.json`, puis sur **Aperçu** (Preview), montre les fiches reçues.

C'est l'onglet à ouvrir en premier quand la page reste vide : il dit si le fichier est arrivé.

Les quatre états

Une page qui charge des données passe par quatre états. Chacun doit montrer quelque chose à la personne qui attend :

État	Quand	Ce que montre Phare
Chargement	Entre la demande et la réponse	« Chargement des lieux… »
Affichée	Les fiches sont arrivées	Les cartes, sans message
Erreur	Le fichier manque, ou il est cassé	« Impossible de charger les lieux. Rechargez la page dans un moment. »
Vide	Le fichier est arrivé, sans aucune fiche	« Aucun lieu pour le moment. »

Le message s'écrit dans un paragraphe `<p id="etat" role="status">`. `role="status"` fait lire le message par les lecteurs d'écran quand il change, sans déplacer le focus. Deux outils de JavaScript servent ici :

- **`.catch`**, à la fin du bloc : « si quelque chose a échoué plus haut, fais ceci ». Il écrit le message d'erreur ;
- **`reponse.ok`** : vrai si le code est un succès. `fetch` ne compte pas un `404` comme un échec ; c'est vous qui le vérifiez, dans le premier `.then`.

Comment savoir que c'est juste

- L'adresse commence par `http://127.0.0.1` en local, par votre adresse GitHub Pages en ligne.
- L'onglet Réseau montre `data.json` avec le code `200`.
- Une carte par fiche : dans la console, `tousLesLieux.length` donne le nombre de fiches de `data.json`.
- Changez une fiche dans `data.json` : sa carte change, sans toucher au HTML.
- Avec un nom de fichier faux, la page dit qu'elle n'a pas pu charger. Elle ne reste pas blanche.

Vérifiez-vous

1. Vous ouvrez `index.html` par un double-clic. L'adresse commence par `file://` et aucune carte n'apparaît. Que faites-vous ?
2. L'onglet Réseau montre `data.json` avec le code `404`. Que vérifiez-vous en premier ?
3. L'onglet Réseau montre `data.json` avec le code `200`, mais la console dit `SyntaxError`. Où est le problème ?
4. Pourquoi le message « Chargement… » ne suffit-il pas, sans `.catch` ?

Les exercices de la semaine

Exercice	Quand	Dans le pack

1 — Phare charge ses fiches : Live Server, `fetch`, l'onglet Réseau, 30 cartes	P1, 30 min, seul	`02-exercices/s11/1-phare-charge-ses-fiches.pdf` et le dossier `02-exercices/s11/1-phare-charge-ses-fiches/depart/`
2 — Les quatre états : provoquer le chargement lent, le fichier absent, cassé et vide, puis les afficher	P2, 30 min, seul	`02-exercices/s11/2-les-quatre-etats.pdf` et le dossier `02-exercices/s11/2-les-quatre-etats/depart/`
3 — Dans votre projet : vos cartes depuis votre `data.json`, en ligne	P2 (fin) et P3, 45 min, avec votre projet	`02-exercices/s11/3-dans-votre-projet.pdf` et le dossier `02-exercices/s11/3-dans-votre-projet/depart/`

Les trois exercices ont deux parcours. Le parcours **essentiel** suit les étapes. Le parcours **avancé** fait les mêmes étapes, puis la partie « Pour les rapides » de chaque énoncé.

Avec l'IA

Exercices 1 et 2 : pas d'IA pour écrire. Les lignes sont courtes, et vous devez les avoir lues une fois pour vérifier celles de l'IA. Exercice 3 : l'IA adapte `afficherLieux` à **vos** cartes. Vous lui donnez une carte de votre maquette et le tableau qui relie chaque texte à une étiquette ; elle ne change rien d'autre. Une IA écrit volontiers `innerHTML`, `async` ou une bibliothèque : refusez, et redemandez avec « Ne change que la fonction afficherLieux ».

Pour les rapides

Chaque énoncé se termine par sa partie « Pour les rapides » : le résumé dans les cartes et l'ordre des messages (exercice 1), l'erreur muette et Firefox (exercice 2), le compte des fiches et la fiche modifiée en ligne (exercice 3). Elle n'est pas notée en plus. La version approfondie, `s11-fetch-avance.pdf`, montre la même lecture écrite avec `await`.

Lien avec la pratique

- **Semaine 12** : la recherche et le filtre travaillent sur `tousLesLieux`, la pile remplie cette semaine. On ne redemande pas `data.json` à chaque lettre tapée.
- **Note 3** : c'est une condition du projet. Les cartes doivent venir d'un `data.json` personnel chargé par `fetch`, et une fiche modifiée doit changer sa carte.
- **Dans le métier**, presque toutes les applications que vous utiliserez chargent leur contenu ainsi : horaires, météo, catalogue. Quand une page reste blanche chez un client, l'onglet Réseau est la première chose que vous ouvrirez.